



**B.Sc. (General/Special) Degree  
Programme  
Faculty of Applied Sciences  
University of Sri Jayewardenepura**

**ICT 1022 Computer System Architecture**

**Presented By:**

**Surani Tissera(PhD)**

**Department of Computer Science**

# Introducion

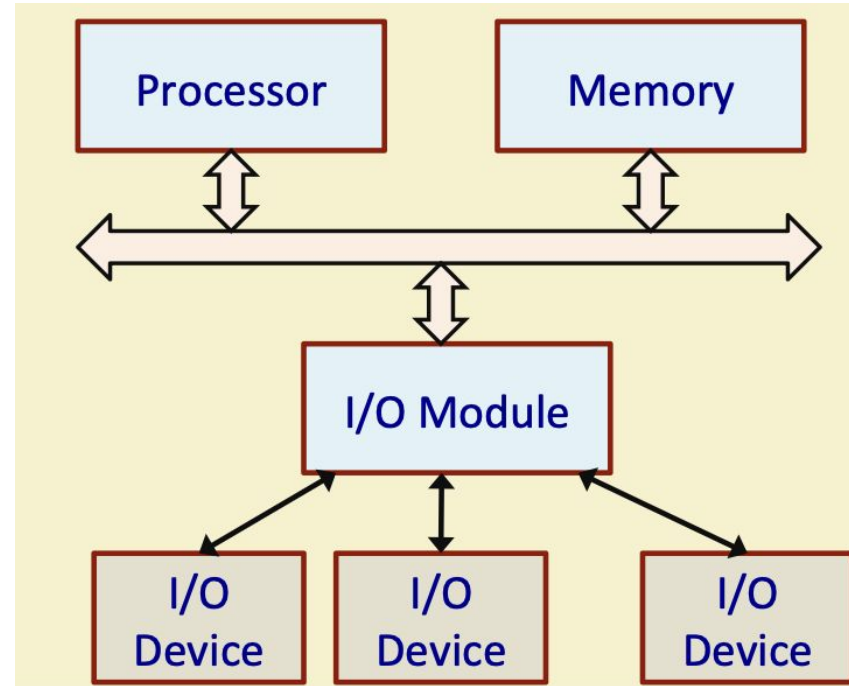
- Interfacing input/output devices is more complex as compared to interfacing memory systems.
- Why?
  - Wide variety of peripherals (keyboard, mouse, disk, camera, printer, scanner,etc.).
  - Widely Varying Speeds.
  - Data transfer rate can be regular or irregular.
  - Size Of Data blocks transferred at a time varies widely (few bytes to Kbytes).
- Slower than processor and memory.



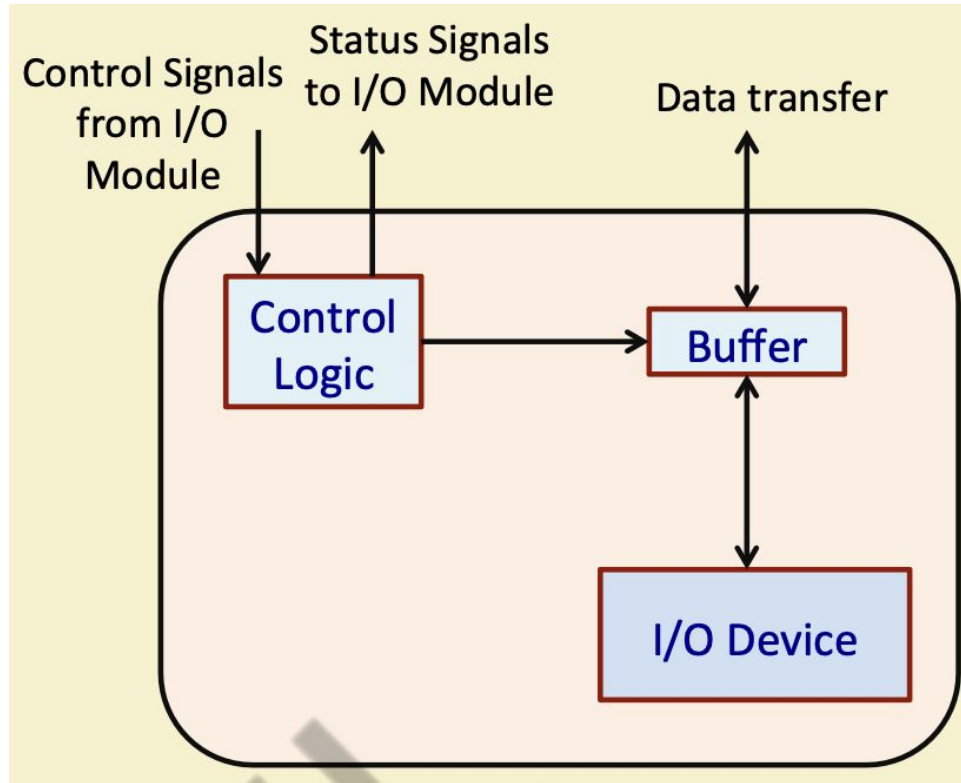
# Input / Output Interface

To handle widely different types of I/O devices, we need a programmable I/O interface or I/O module.

- Interfaces to processor and memory on one side.
- Interfaces to one or more peripheral devices on the other side.

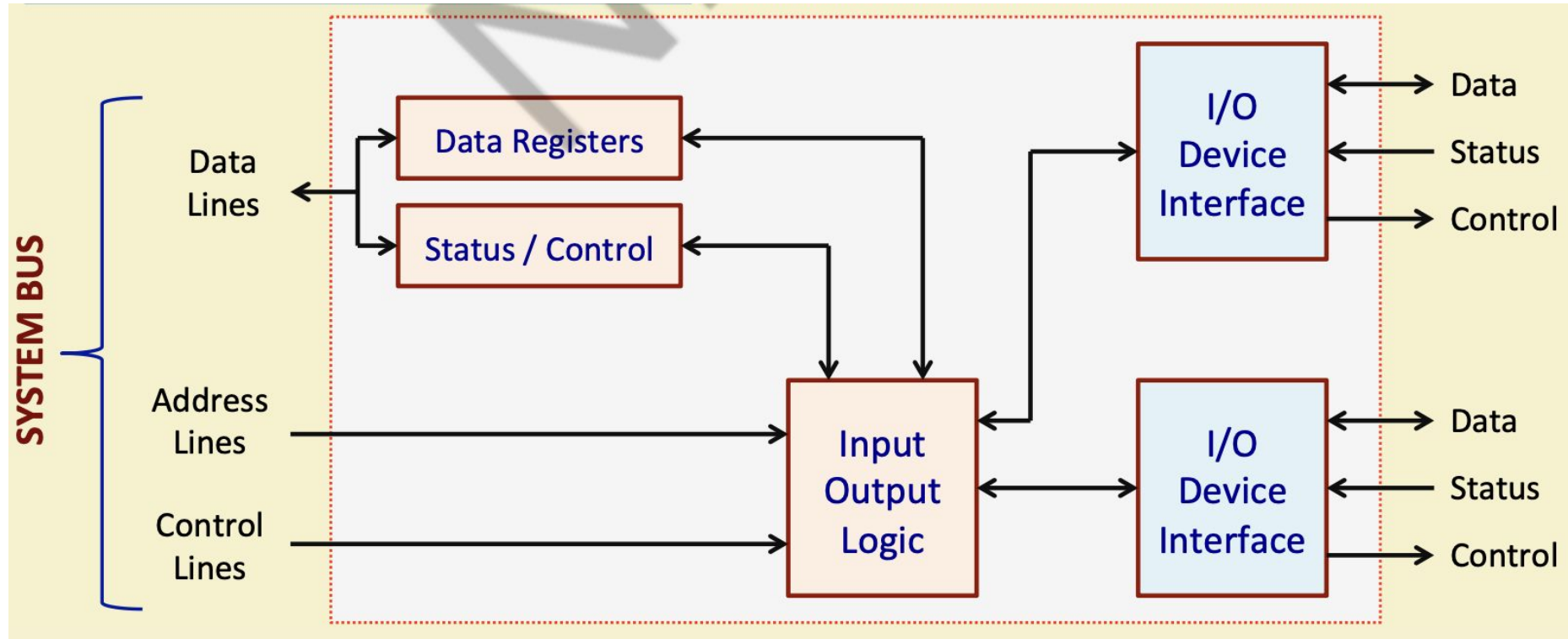


# Typical I/O Device Interface



This is a property of Department of Computer Science, Faculty of Applied Science, University of Sri Jayewardenepura.

# I/O Module Schematic



# Typical Steps During I/O

- a) Processor requests the I/O Module for device status.
- b) I/O Module returns the status to the processor.
- c) If the device is ready, processor requests data transfer.
- d) I/O Module gets data from device (say, input device).
- e) I/O Module transfers data to the processor.
- f) Processor stores the data in memory.

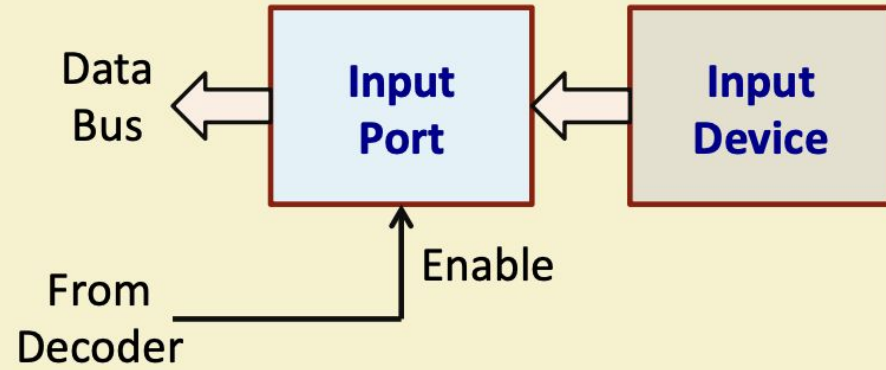
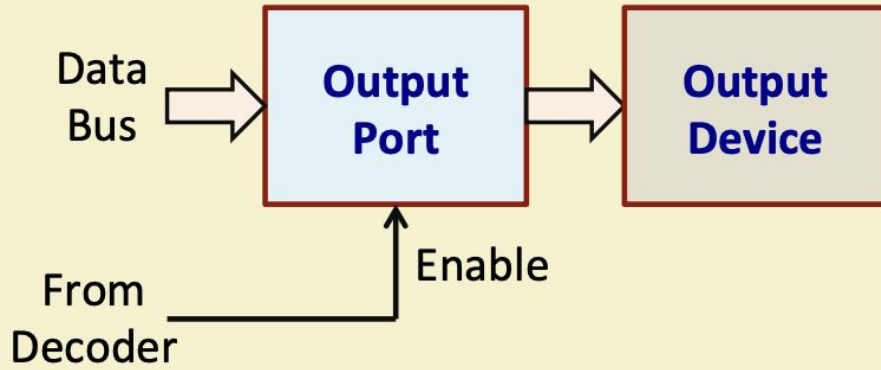


# How are I/O devices typically interfaced?

- Through input and output ports.
- Output port:
  - Basically a PIPO register that is enabled when a particular output device address is given.
  - The register inputs are connected to the data bus, and the register outputs are connected to the output device.
- Input port:
  - Basically a parallel tri state bus driver that is enabled when a particular Input device address is given.
  - The driver outputs are connected to the databus, while the inputs are connected to the input device.



# Example of output and input ports



# Memory-Mapped and I/O Mapped Device Interface

- Memory-mapped device interface:
  - The same address decoder selects memory and I/O ports.
  - Some of the memory address space is occupied by I/O devices.
  - All data transfer instructions to/from memory can be used to transfer data to/from I/O devices.
  - The processor need not have separate instructions for I/O, nor it need to specify whether an address generated by the CPU is a memory address or an I/O address.



# Memory-Mapped and I/O Mapped Device Interface

- I/O mapped device interface:
  - Separate instructions for I/O data transfer (say, IN and OUT).
  - A processor signal identifies whether a generated address refers to a memory location or an I/O device.
  - Separate address decoders for selecting memory and I/O ports.
  - The complete memory address space can be utilized.



# DIRECT MEMORY ACCESS



# Introduction

In the data transfer methods discussed under programmed I/O, it is assumed that machine instructions are used to transfer the data between I/O device and memory.

- Not very suitable when large blocks of data are required to be transferred at high speed (e.g. transfer of a disk block).
- An alternate approach is Direct Memory Access (DMA).
  - Allows transfer of a block of data directly between an I/O device and memory, without continuous CPU intervention.



- Why programmed I/O is not suitable for high-speed data transfer?
  - a) Several program instructions have to be executed for each data word transferred between the I/O device and memory.
    - Suppose 20 instructions are required for each word transfer.
    - The CPI of the machine running at 1 GHz clock is 1.
    - So, 20 nsec is required for each word transfer à maximum 50 M words/sec
    - Data transfer rates of fast disks are higher than this figure.



b) Many high speed peripheral devices like disk have a synchronous mode of operation, where data are transferred at a fixed rate.

- Consider a disk rotating at 7200 rpm, with average rotational delay of 4.15 msec.
- Suppose there are 64 Kbytes of data recorded in every track.
- Once the disk head reaches the desired track, there will be a sustained data transfer at rate  $64 \text{ Kbytes} / 4.15 \text{ msec} = 15.4 \text{ MBps}$ .
- This sustained data transfer rate is comparable to the memory bandwidth, and cannot be handled by programmed I/O.

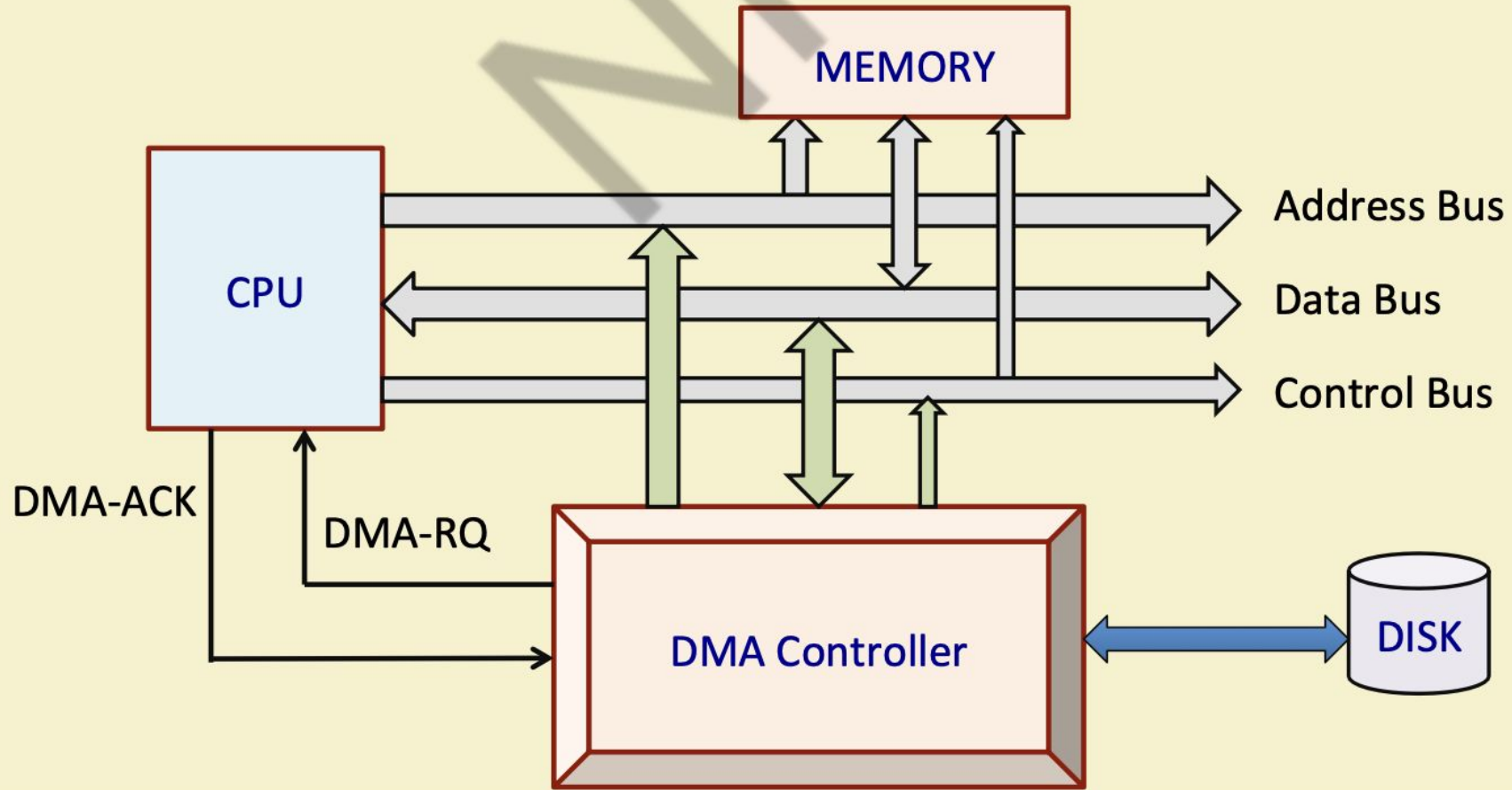


# DMA Controller

A hardwired controller called the DMA controller can enable direct data transfer between I/O device (e.g. disk) and memory without CPU intervention.

- No need to execute instructions to carry out data transfer.
- Maximum data transfer speed will be determined by the rate with which memory read and write operations can be carried out.
- Much faster than programmed I/O.



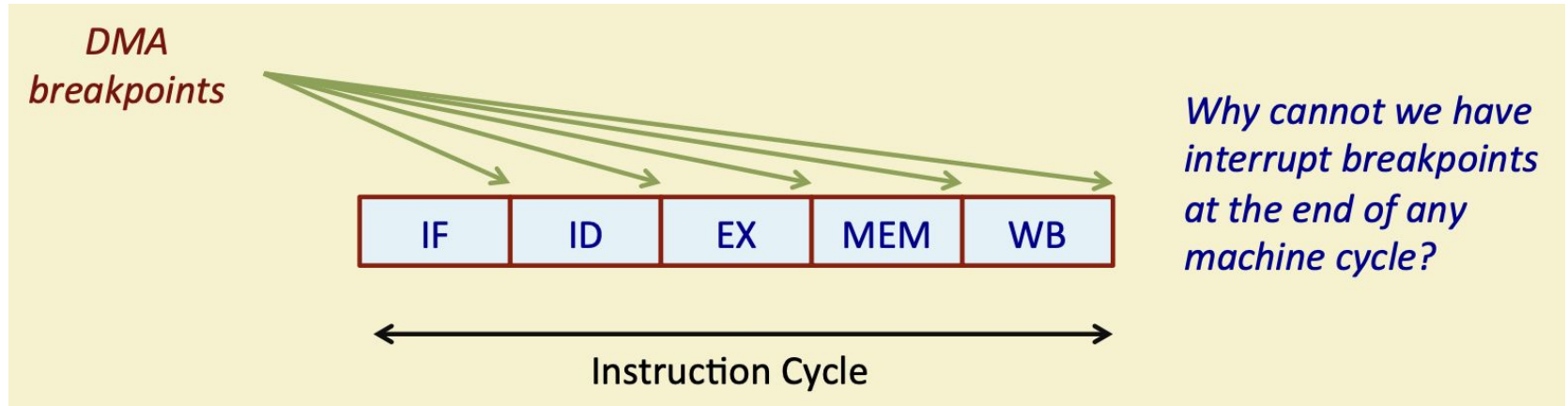


# Steps Involved

- When the CPU wants to transfer data, it initializes the DMA controller.
  - How many bytes to transfer, address in memory for the transfer.
- When the I/O device is ready for the transfer, the DMA controller sends DMA-RQ signal to the CPU.
- CPU waits till the next DMA breakpoint, relinquishes control of the bus (i.e. puts them in high impedance state), and sends DMA-ACK to DMA controller.
- Now DMA controller enables its bus interface, and transfers data directly to/from memory.
- When done, it deactivates the DMA-RQ signal.
- The CPU again begins to use the bus to access memory.

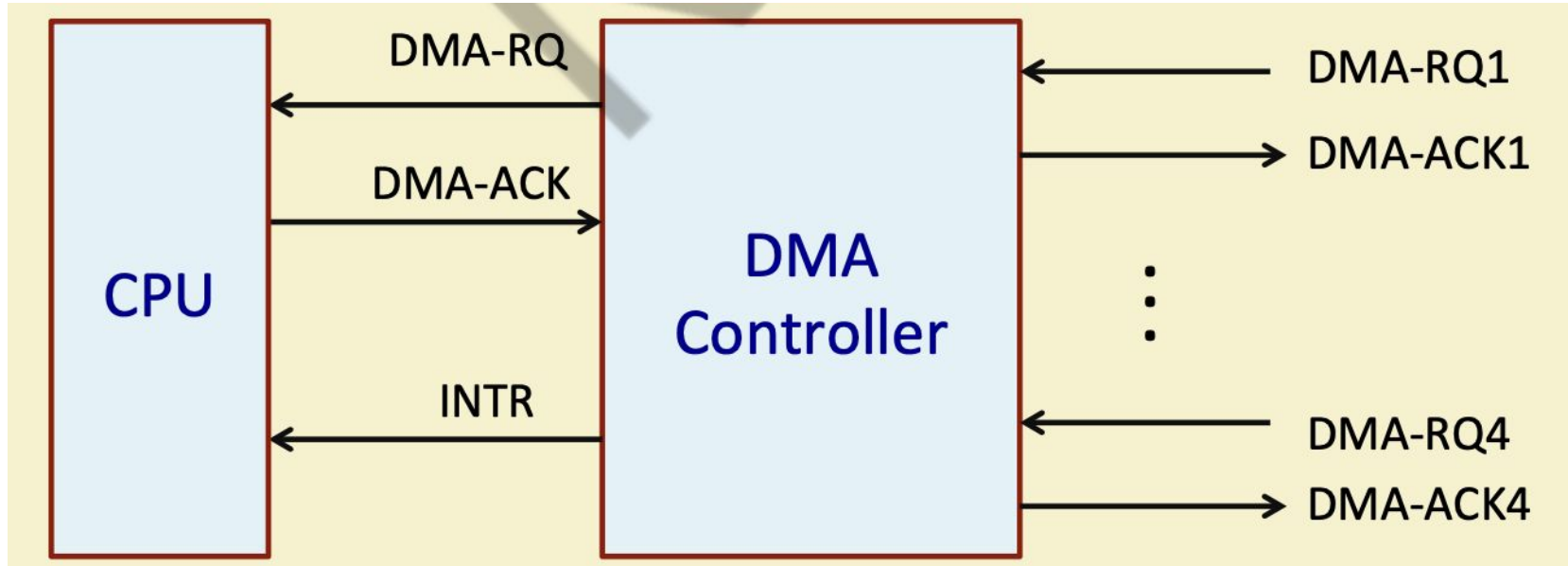


- The DMA breakpoints:
  - DMA request can be acknowledged at the end of any machine cycle.



- For every DMA channel, the DMA controller will have three registers:
  - a) Memory address
  - b) Word count
  - c) Address of data on disk
- CPU initializes these registers before each DMA transfer operation.
- Before the data transfer, DMA controller requests the memory bus from the CPU.
- When the data transfer is complete, the DMA controller sends an interrupt signal to the CPU.





# DMA Transfer Modes

- DMA transfer can take place in two modes:

## a) DMA cycle stealing

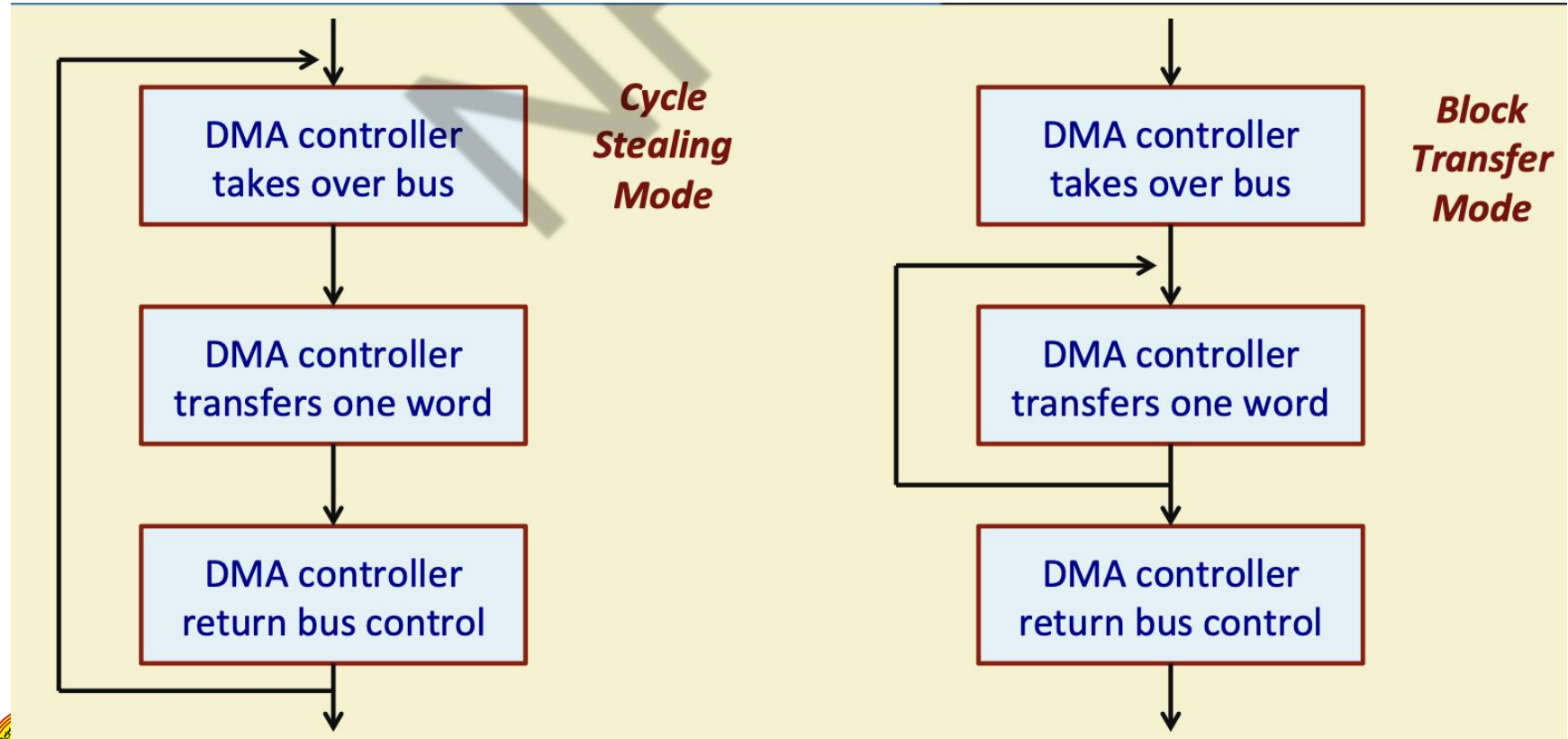
- The DMA controller requests for the for a few cycles 1 or 2.
- Preferably when the CPU is not using memory.
- DMA controller is said to steal cycles from the CPU without the CPU knowing it.

## b) DMA block transfer

- The DMA controller transfers the whole block of data without interruption.
- Results in maximum possible data transfer rate.
- CPU will lie idle during this period as it cannot fetch any instructions from memory.



# DMA Transfer Modes



# Others Applications of DMA

Other than data transfer to/from high-speed peripheral devices, DMA can be used in some other areas as well:

- High-speed memory-to-memory block move.
- Refreshing dynamic memory systems, by periodically generating dummy read requests to the columns.



# Others Applications of DMA

Other than data transfer to/from high-speed peripheral devices, DMA can be used in some other areas as well:

- High-speed memory-to-memory block move.
- Refreshing dynamic memory systems, by periodically generating dummy read requests to the columns.



Thank  
you!

dreamstime

